

SQL DML AND DATA INTEGRITY IN PRACTICE

CHANGING DATA SAFELY

Lecture 6 – Databases and Software Architecture – Autumn 2026

AALBORG
UNIVERSITY

BEFORE WE START: SOME CHANGES TO THIS COURSE

WHY WE ARE CHANGING THINGS: AI

1. **AI is changing the world** – how we live, how we learn, and how we work.
2. **So some ways of teaching are going out of date.** Lectures 1–5 taught what a database course has always taught: how to **code** – `CREATE TABLE`, `SELECT`, `JOIN`.
3. **But coding was never the goal.** It is a **means** – a way to reach understanding.

What should you understand and what is AI taking over?

ON THE SURFACE, AI IS EATING EVERYTHING

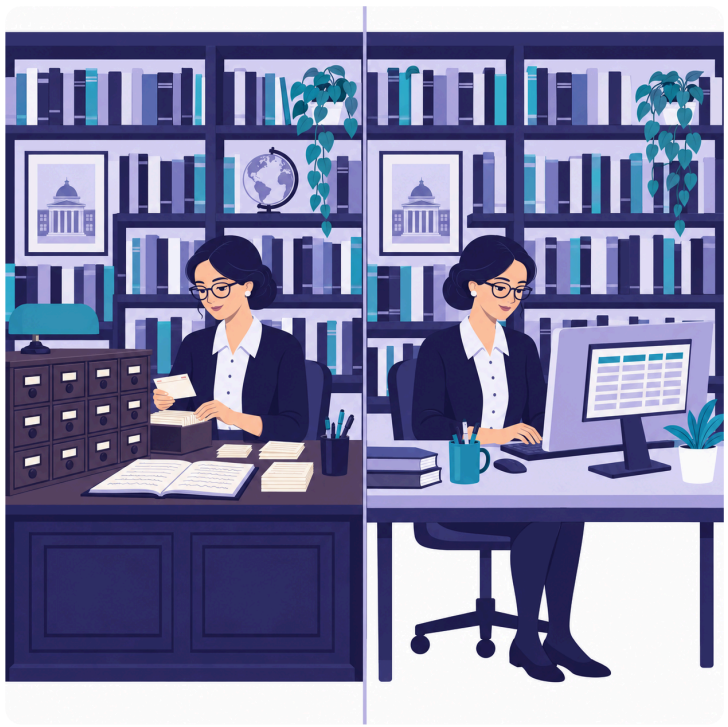


Look closer. What AI eats is the **surface** of every task:

- the **execution** – typing it out
- the **repetitive** part – the boilerplate, the tenth `INSERT`
- everything that **can be written down completely** – once you can say exactly what you want, a machine can do it

What it hands back to you is the core – heavier than before, because now it is **all** that is left: knowing what to want, and knowing whether you got it.

EXAMPLE 1: THE LIBRARIAN, BEFORE AND AFTER



CORE SKILL

Understanding the library

its entities (books, copies, members, loans), how they relate, and what every operation means: lend, return, reserve, withdraw

BEFORE

Records kept by hand

card catalogues, ledgers, index cards

AFTER

Records kept in a database

tables, keys, queries

The information revolution changed the tool – not the understanding of the library.

EXAMPLE 2: THE DATABASE ENGINEER, BEFORE AND AFTER AI



CORE SKILL

Understanding the data

its entities and relationships, every kind of operation on it, and the experience to keep it safe and stable

BEFORE

Writing SQL and code by hand

every table, query and transaction typed out

AFTER

Specifying and judging

say precisely what is needed; check what an AI writes

AI is changing how the code gets written – not the understanding of the data.

SO THIS COURSE SEPARATES THE TWO

From here on, we keep two things apart, so you always know which one you are looking at.

WHAT LASTS

Problems, understanding and judgment

What a database exists to solve, and how to decide well – the part to keep in your head.

WHAT CHANGES

The tool of the day

The SQL, the syntax, the code – which an AI can increasingly write for you, once you can say what you need.

To make the separation visible, I have built a **mind map** of the course.

ONE MIND MAP, THREE PERSPECTIVES

Perspective	What it holds
1 · Key points	Understanding of data: the real problems and scenarios that come with it, and how today's databases deal with such data in general. The tools may change; these problems will not
2 · Example	One running example, the library, used throughout the course. It is the bridge that connects the key points and the database practice to a real-world scenario, so both are easier to understand
3 · Practice	The concrete practice: how to create the tables, how to write the SQL

Every slide from here on carries one of these three labels in its top-right corner.

THE SHAPE OF THE COURSE

Lectures	Content
1–8	Databases – this mind map covers Lectures 1–8
9–11	Software architecture – teaching materials released later this month
12	Questions and answers

Now, let us open the mind map of first part ‘Databases’ and walk through it.

BACK TO LECTURE 6: SQL DML AND DATA INTEGRITY IN PRACTICE – CHANGING DATA SAFELY

FROM READING DATA TO CHANGING IT – SAFELY

Until now we have only **read** data, with `SELECT`. Reading never changes anything, so a mistake costs nothing. Today we start **changing** data. We follow the life of one record – it arrives, it changes, it leaves – and at each step ask a question that reading never did:

NEED 1 **It arrives – at the desk, or in a file.**
It may already exist: whose data wins? And what about rows nobody can decode?

NEED 2 **It changes – and a typo hits every row.**
How do you change exactly what you mean, and undo it when you don't?

NEED 3 **It leaves.**
Delete it, or only mark it – and lose its history or keep it?

NEED 4 **It leaves, but others still point to it.**
What happens to a member's loans, and to the members they referred?

RECAP: THE LIBRARY SCHEMA

Same seven tables as every lecture since Lecture 3, unchanged: `Author`, `Book`, `Member`, `BookGenre`, `Copy`, `BookAuthor`, `Loan` – plus `Member.referred_by`, added in Lecture 5, which plays a starring role today. Every live demo uses this exact schema and seed data.

- `Book(book_id, title, isbn, publication_year)`
- `Author(author_id, first_name, last_name, birth_year)`
- `Member(member_id, first_name, last_name, email, join_date, referred_by)`
- `Copy(book_id, copy_number, status)` – weak entity, `status` is `'available'` or `'on loan'`
- `Loan(loan_id, member_id, book_id, copy_number, loan_date, due_date, return_date)`
- Plus `BookGenre`, `BookAuthor` – junction tables linking the above

NEED 1: SOMETHING NEW ARRIVES

A new member, Lisa, signs up at the desk. The city library sends its member list.
A spreadsheet from 2009 must be brought in.
SQL: `INSERT`, `RETURNING`, `ON CONFLICT`, `COPY`

NEED 1: SOMETHING NEW ARRIVES

Something new arrives: one record at the desk, or many at once from a list or a file.

WHAT THE DATABASE CAN PROVIDE FOR YOU

1. Add one record

Lisa signs up at the desk. The database gives her a member number that no one else has, and hands it back at once.

2. Add many records at once

A whole list or file goes in with one command. Nobody types the rows one by one.

WHAT YOU MUST DECIDE

3. What if it is already there?

The city sends “Anna Nielsen, new address”. We already have Anna. Keep our address, or take theirs?

4. What if you cannot read it?

An old file says `03/04/2024`. Is that 3 April or 4 March? Do not guess: set the row aside and ask.

IN THE LIBRARY: LISA, ANNA, AND AN OLD FILE

The same four, as they happen in our library:

1. **One new member.** Lisa wants a library card. The desk adds her, and the database gives her a member number at once, so the card can be printed.
2. **Many records at once.** In spring, every book from before 1950 gets the tag *Classic* – all of them in one step.
3. **Anna is already here.** The city library sends its list of members. Most are already in our database – Anna Nielsen, for example, with a new address. One repeated name must not stop the whole list, and we must not end up with two Annas.
4. **A file we cannot fully read.** An old file from 2009 arrives. Danish letters such as *ø* show up broken. Dates are written in three different ways. Where there is no email, it says `NULL`.

WHAT THE DATABASE OFFERS FOR NEED 1

	Need	SQL	What it does
1.1	Add one record	`INSERT`	One new row, with values for the columns you name
1.2	Add many at once	multi-row `INSERT`, `INSERT ... SELECT`, `COPY`	Many rows in one command: from a list, from another table, or from a file
1.3	It is already there	`ON CONFLICT`	You say in advance what to do: skip the row, or update it
1.4	A file from elsewhere	`COPY ... WITH (ENCODING ..., NULL ...)`	You tell the database the file's rules – its letters and how it writes "empty" – and it reads them correctly

Under 1.1 and 1.2: `SERIAL` is the foundation – it picks each new row's number. `RETURNING` saves a second query by handing that number back.

What it cannot do for you: decide whose data is right (1.3), or read `03/04/2024` (1.4). Put such rows in a separate table and ask the sender. Never guess.

INSERT: SINGLE ROW

```sql
INSERT INTO Author (first_name, last_name, birth_year)
VALUES ('Ursula', 'Le Guin', 1929);
```

Read it as: *put one new row into `Author`, with these values for these columns.*

- **Name the columns, then give the values in the same order.** Without the column list, the values must follow the table's own column order – and the statement breaks when someone adds a column later.
- **A column you leave out gets its default value.** With no default, it gets `NULL`. If it is also `NOT NULL`, the insert fails.

INSERT: MULTIPLE ROWS IN ONE STATEMENT

```
INSERT INTO Author (first_name, last_name, birth_year) VALUES
  ('Ursula', 'Le Guin', 1929),
  ('Octavia', 'Butler', 1947),
  ('Ted', 'Chiang', 1967);
```

```
INSERT 0 3
```

Read it as: *put three new rows into `Author`, with one statement.*

- **List several rows after VALUES, separated by commas.** `INSERT 0 3` means three rows were added. (The `0` is a leftover field and is always 0.)
- **Faster than three separate INSERTs.** Every statement is a round trip: send it, wait, get the answer. One trip instead of three adds up when you load real data.
- **All or nothing.** If one of the rows is wrong, none of them is added.

INSERT ... SELECT: COPYING DATA BETWEEN TABLES

```sql
INSERT INTO BookGenre (book_id, genre)
SELECT book_id, 'Classic' FROM Book
WHERE publication_year < 1950;
```

Read it as: *for every book from before 1950, add the tag Classic.*

- **The rows come from a query, not from typed values.** The `SELECT` finds the books, and the `INSERT` adds one row for each. The data never leaves the database.
- **One statement, any number of rows** – 20 books or 2,000, the statement is the same.
- **One duplicate stops everything.** 17 of these books are already tagged Classic, and `BookGenre`'s key `(book_id, genre)` allows each pair only once. So this statement fails and adds nothing. `ON CONFLICT` (Need 1.3) fixes that.

SERIAL AND AUTO-GENERATED KEYS

```
CREATE TABLE Author (
    author_id   SERIAL PRIMARY KEY,
    ...
);
```

Read it as: *author_id is a number the database fills in by itself.*

- **author_id is left out on purpose.** None of the INSERTs so far mentions it. The database gives each new row the next number: 43, 44, 45, …
- **Each number is used only once.** Two inserts at the same moment never get the same number, and a number is not handed out again – not even after a DELETE or a ROLLBACK.
- **SERIAL is a shortcut.** It means an INTEGER column plus a counter (a *sequence*) that hands out the next value. Newer code often writes GENERATED ALWAYS AS IDENTITY, the SQL-standard form; this course uses SERIAL.

THE RETURNING CLAUSE

Lisa's card needs her new member number right away. RETURNING gives it back in the same statement.

```
INSERT INTO Member (first_name, last_name, email)
VALUES ('Lisa', 'Berg', 'lisa@example.com')
RETURNING member_id, join_date;
```

member_id	join_date
51	*today*

- **No second query.** Without RETURNING you insert, then run a SELECT to find the number – two trips to the database instead of one.
- **Any column, not just the key.** We never typed join_date: the database filled in today's date as the default, and RETURNING shows it.
- **Works for many rows too.** A multi-row INSERT or an INSERT ... SELECT returns one line per new row. It also works on UPDATE and DELETE, to see exactly which rows were changed or removed.

ON CONFLICT DO NOTHING: SKIP WHAT IS ALREADY THERE

```
INSERT INTO BookGenre (book_id, genre)
SELECT book_id, 'Classic' FROM Book WHERE publication_year < 1950
ON CONFLICT (book_id, genre) DO NOTHING;      -- INSERT 0 3
```

Read it as: *tag every book from before 1950 as Classic – and skip the ones that already have the tag.*

- **Plain `INSERT` stops at the first duplicate.** 17 of these 20 books are already tagged Classic, so the plain `INSERT ... SELECT` fails and adds nothing.
- **`ON CONFLICT` says what to do instead.** `(book_id, genre)` is the key to check. `DO NOTHING` means: skip that row and carry on. Only the 3 missing tags are added.
- **Keep what is there, or change it?** `DO NOTHING` keeps the row that is already there. `DO UPDATE`, on the next slide, changes it.

ON CONFLICT DO UPDATE: UPSERT

```sql
INSERT INTO Member (first_name, last_name, email, join_date)
VALUES ('Daniel', 'Petersen', 'daniel.petersen@example.dk', '2019-05-02')
ON CONFLICT (email)
DO UPDATE SET join_date = EXCLUDED.join_date;
```

Read it as: *add Daniel – and if his email is already here, take the city's `join_date` instead.* "Upsert" = update + insert: insert the row, or update it if it is already there.

- `EXCLUDED` **is the incoming row** – the one that could not be inserted. `EXCLUDED.join_date` is the city's date.
- **Only the columns you list change.** Here that is `join_date`; everything else stays ours.
- **The decision: whose data wins, column by column.** Is the city's `join_date` the day Daniel joined *us*? No – so for this column `DO UPDATE` is the wrong choice. A corrected surname from the city? Probably theirs.

IMPORTING DATA: COPY

```
COPY Member (first_name, last_name, email, join_date)
FROM '/path/to/new_members.csv'
WITH (FORMAT csv, HEADER true);
```

Read it as: *load every row of this file into `Member`, in one go.*

- **Much faster than one `INSERT` per row.** This is the way to load a whole file.
- `HEADER true` **skips the first line**, which holds the column names.
- `COPY` **reads the file on the database server.** For a file on your own laptop, use `psql`'s `\copy`: the same options, but it reads the file where `psql` runs.
- **Real files are rarely this clean** – next slide.

TYPICAL CSV PROBLEMS

```
first_name,last_name,email,join_date
Anna,Nielsen,anna@example.com,2024-03-15
S<f8>ren,Kj<e6>r,soren@example.com,15/03/2024
Line,Thomsen,NULL,2024-03-16
Mikkel,,mikkel@example.com,
Nikolaj,Lund,nikolaj@example.com,03/04/2024
```

Four of these five rows have a problem:

- **Row 2: Danish letters in another encoding.** `<f8>` is ø and `<e6>` is æ in Latin-1. The database expects UTF-8, so `COPY` stops with an error.
- **Row 3: the text `NULL` is four letters, not a missing value.** A search for members without an email would miss Line.
- **Row 4: empty fields.** `last_name` is `NOT NULL`, so this row cannot go in as it is.
- **Row 5: `03/04/2024`.** 3 April or 4 March? The file cannot tell you.

FIXING IT: WHAT COPY OFFERS, AND WHAT COMES AFTER

```
CREATE TABLE staging_members (first_name TEXT, last_name TEXT, email TEXT, join_date TEXT);
COPY staging_members FROM '/path/to/new_members.csv'
WITH (FORMAT csv, HEADER true, ENCODING 'LATIN1', NULL 'NULL');
```

Read it as: *load the file into a separate table first, and tell COPY the file's rules.*

- `ENCODING 'LATIN1'` – read the bytes as Latin-1, so ø and æ arrive correctly.
- `NULL 'NULL'` – treat the text `NULL` as a real missing value. Careful: an empty field is then no longer `NULL`, but an empty text.
- **Options fix the whole file, not single rows.** Every column is `TEXT`, so no odd date or missing name can stop the load. Fix what is clear, then move those rows into `Member`.
- **The decision:** no option can fix `03/04/2024`. Set the row aside and ask the sender. Never guess.

INDEX: NEED 1

The need	Real scenarios	What the database provides	Read more
1.1 Add one record	Lisa gets a card; the printer needs her number now	`INSERT`, with `SERIAL` and `RETURNING` `INSERT INTO Member (...) VALUES (...) RETURNING member_id;`	DBS 3.9
1.2 Add many at once	Three new authors arrive at once	multi-row `INSERT` `INSERT ... VALUES (...), (...), (...);`	DBS 3.9
	Tag every book from before 1950 as Classic	`INSERT ... SELECT` `INSERT INTO BookGenre SELECT ... FROM Book WHERE ...;`	DBS 3.9
1.3 It is already there	Load the city's member list without duplicates	upsert – whose data wins, per column `ON CONFLICT (...) DO NOTHING` / `DO UPDATE SET ...`	PG `INSERT`
1.4 A file from elsewhere	The city's 2009 member list, as CSV	bulk load; encoding; how "empty" is written `COPY t FROM '...' WITH (ENCODING 'LATIN1', NULL 'NULL');`	PG `COPY`
	`03/04/2024` – 3 April or 4 March?	set aside and ask, never guess a table of `TEXT` columns; send unclear rows back	exercise sheet

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (3.9 = chapter 3, section 9); PG = the PostgreSQL documentation, page for that command.

BREAK

5-MINUTE BREAK

Stretch, grab water. Next: the record is in – now it changes. And what happens when a colleague changes data but forgets to say which rows.

NEED 2: SOMETHING CHANGES

A copy of *Dune* is lost, so its record must change. A colleague changes data but forgets to say which rows.

SQL: `UPDATE`, with `BEGIN` as a safety net

NEED 2: SOMETHING CHANGES

A record does not stay the same: a copy is lost, a member moves, a loan is returned. Every time, three questions come up.

WHAT THE DATABASE CAN PROVIDE FOR YOU

1. Change the rows you name

One copy of *Dune* is lost. Exactly the rows your condition matches change – no more, no fewer.

2. Rows found through another table

Every copy of *Dune*, found by its title in another table.

WHAT YOU MUST DECIDE

3. Undo a mistake

A colleague forgets the condition, and every copy is now lost. The database cannot tell that this was wrong. Only you can see the number is wrong – check it before you make the change final.

IN THE LIBRARY: “MARK IT LOST”

The same three, as they happen in our library:

1. **One copy is lost.** A member reports that copy 1 of *Dune* is gone. The librarian marks it lost – one change to one row.
2. **Every copy of a title.** The second copy turns up damaged: “mark every copy of *Dune* lost”. The copies are found by the book’s title, not by their number.
3. **A missing condition.** A colleague types the same change but forgets to say which rows. Every copy in the building is now lost. Can that be undone?

WHAT THE DATABASE OFFERS FOR NEED 2

	Need	SQL	What it does
2.1	Change the rows you name	`UPDATE ... SET ... WHERE`	Changes exactly the rows the `WHERE` matches. No `WHERE`: every row
2.2	Rows found through another table	`UPDATE ... FROM`	Joins in the table that knows, for example, the title
2.3	Undo a mistake	`BEGIN ... ROLLBACK`	Nothing is final until `COMMIT`: check the row count first, then decide

What it cannot do for you: notice that you forgot the `WHERE` (2.3). It changes what you tell it to.

UPDATE: SET AND WHERE

```
UPDATE Copy
SET status = 'lost'
WHERE book_id = 16 AND copy_number = 1;
```

Read it as: *in `Copy`, set the status to lost – but only for copy 1 of book 16.*

- **`SET` says what changes; `WHERE` says which rows.** The `WHERE` works exactly as in a `SELECT`.
- **Try the same `WHERE` in a `SELECT` first,** to see which rows it will change.
- **Leave out the `WHERE`, and every row changes** – the live demo in two slides.

UPDATE ... FROM: UPDATING BASED ON A JOIN

```
UPDATE Copy c
SET status = 'lost'
FROM Book b
WHERE c.book_id = b.book_id AND b.title = 'Dune';
-- UPDATE 2
```

book_id	copy_number	status
16	1	lost
16	2	lost

Read it as: *mark every copy of Dune as lost – find the copies through `Book`, by title.*

- `Copy` **has no title.** `FROM Book` brings in the table that has it.
- **The join condition goes in the `WHERE`:** `c.book_id = b.book_id`. Forget it, and every copy matches.

UPDATE WITHOUT WHERE

```
BEGIN;
UPDATE Copy SET status = 'lost';
SELECT count(*) FROM Copy WHERE status = 'lost';
```

```
UPDATE 157
 count
-------
   157
```

Read it as: *start a draft with BEGIN, change every copy to lost, then count.*

- **BEGIN starts a draft.** From here on, nothing is final until you say so.
- **The UPDATE has no WHERE, so it changes every row:** `UPDATE 157`. Every copy is now lost, available and on loan alike.
- **The number is the warning sign.** You meant one copy; the database says 157. Next slide: how to get out.

UNDO IT WITH ROLLBACK

Get out: throw the draft away

```
ROLLBACK;
SELECT count(*) FROM Copy
WHERE status = 'lost';
```

```
ROLLBACK
 count
-------
     3
```

Next time: the safe routine

```
BEGIN;
UPDATE Copy SET status = 'lost'
WHERE book_id = 16 AND copy_number = 1;
-- UPDATE 1  <- the number you expected?
COMMIT;      -- yes: make it final
-- ROLLBACK; -- no: throw it away
```

- ROLLBACK **undoes everything since** BEGIN. Back to 3 lost copies – the ones that were lost before the demo. The other 154 are as they were.
- **Without** BEGIN**, every statement is final the moment it runs.** Then there is nothing to roll back – only a backup can help.
- **The habit:** BEGIN before any UPDATE or DELETE you type by hand. Check the number, then COMMIT or ROLLBACK. What a transaction really is: next lecture.

INDEX: NEED 2

The need	Real scenarios	What the database provides	Read more
2.1 Change the rows you name	Copy 1 of *Dune* is reported lost	`UPDATE ... SET ... WHERE` `UPDATE Copy SET status = 'lost' WHERE book_id = 16 AND copy_number = 1;`	DBS 3.9
2.2 Rows found through another table	Every copy of *Dune*, found by title	`UPDATE ... FROM` (a join in an update) `UPDATE Copy c SET ... FROM Book b WHERE c.book_id = b.book_id AND ...;`	PG `UPDATE`
2.3 Undo a mistake	A colleague forgets the `WHERE`	`BEGIN ... ROLLBACK` as a safety net `BEGIN; UPDATE ...; SELECT count(*) ...; ROLLBACK;`	DBS 4.3

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (3.9 = chapter 3, section 9); PG = the PostgreSQL documentation, page for that command.

NEED 3: SOMETHING LEAVES

Some books are withdrawn and must go. Sometimes a whole table must be emptied.

SQL: `DELETE`, `TRUNCATE`

NEED 3: SOMETHING LEAVES

Records do not stay forever: a book is withdrawn, an account is closed, a test table is wiped. Three questions come up.

WHAT THE DATABASE CAN PROVIDE FOR YOU

1. Remove the rows you name

Two copies that were never borrowed can simply go – exactly those, and no others.

3. Empty a whole table fast

Last week's import table: every row, at once.

WHAT YOU MUST DECIDE

2. Delete it, or only mark it?

A withdrawn copy has years of loans behind it. Delete it, and that history is gone. Mark it, and it leaves the shelf but stays in the records.

IN THE LIBRARY: WITHDRAWN BOOKS

The same three, as they happen in our library:

1. **Remove exactly these rows.** The two copies of *Guards! Guards!* were never borrowed. They can simply be deleted.
2. **Delete or mark?** A shelf of damaged books is withdrawn. Each copy has years of loans behind it: who borrowed it, when, and whether it came back late. Delete the copies, and that history is gone. Mark them `'withdrawn'`, and they stay in the records but leave the shelf.
3. **Empty a whole table.** A table from last week's import must be emptied – every row, fast.

WHAT THE DATABASE OFFERS FOR NEED 3

	Need	SQL	What it does
3.1	Remove the rows you name	`DELETE ... WHERE`	Removes exactly the rows the `WHERE` matches. No `WHERE`: every row
3.2	Delete or mark?	`ON DELETE RESTRICT`; `UPDATE ... SET status`	Refuses to delete rows that others point at – so you mark them instead
3.3	Empty a whole table	`TRUNCATE`	All rows at once, much faster – with its own rule for numbering

What it cannot do for you: decide whether a withdrawn book should be deleted or marked (3.2).

DELETE: WITH AND WITHOUT WHERE

With WHERE: exactly these rows

```
DELETE FROM Copy WHERE book_id = 6;
-- DELETE 2
```

Without WHERE: every row

```
BEGIN;
DELETE FROM BookGenre;
-- DELETE 139: every genre tag
ROLLBACK;
```

- **WHERE picks the rows, as in UPDATE.** Left: the two copies of *Guards! Guards!* (book 6). Right: no WHERE, so all 139 genre tags go – saved only by the ROLLBACK.
- **A foreign key can block the delete.** *Dune*'s copies – even DELETE FROM Copy; as a whole – are refused: Loan still points at them (ON DELETE RESTRICT). Next slide: what to do instead.
- **DELETE ... RETURNING *** shows exactly which rows were removed.

DELETE OR MARK?

Delete: refused

```
DELETE FROM Copy WHERE book_id = 16;
```

```
ERROR:  update or delete on table "copy"
  violates foreign key constraint
  "loan_book_id_copy_number_fkey" on table
"loan"
DETAIL:  Key (book_id, copy_number)=(16, 1)
  is still referenced from table "loan".
```

Mark: the history stays

```
UPDATE Copy SET status = 'withdrawn'
WHERE book_id = 16;
-- UPDATE 2
SELECT count(*) FROM Loan
WHERE book_id = 16;
-- 6 loans, still there
```

Read it as: *Dune is withdrawn. Deleting its copies is refused – so mark them instead.*

- **The refusal is a message.** The copies carry history: 6 loans still point at them.
- **Marking keeps that history.** The copies leave the shelf, and every loan still points at a real copy.
- **Delete only what nobody points at** – like the never-borrowed copies of *Guards! Guards!*.

TRUNCATE: THE IDENTITY-RESET SURPRISE

```
-- demo_seq: id SERIAL, with three rows a, b, c
DELETE FROM demo_seq;              -- 3 rows removed
INSERT INTO demo_seq (val) VALUES ('d');
-- id = 4  (the counter keeps going)

TRUNCATE TABLE demo_seq;           -- no RESTART IDENTITY
INSERT INTO demo_seq (val) VALUES ('e');
-- id = 5, NOT 1 !
```

TRUNCATE empties a whole table in one go. Read it as: *empty the table twice, and watch the next number.*

- **Many people think TRUNCATE resets the counter.** It does not: the next row gets 5, not 1.
- **To start again at 1, say so:** `TRUNCATE TABLE demo_seq RESTART IDENTITY;`

TRUNCATE VS. DELETE: WHAT'S DIFFERENT

So how is `TRUNCATE` different from a `DELETE` without `WHERE`? Faster, and with other rules:

`DELETE` (no `WHERE`)	`TRUNCATE`
Removes the rows one by one	Drops the table's storage at once – much faster
Runs row-level `DELETE` triggers	Row-level triggers do **not** run
Locks the rows it deletes	Locks the whole table
Keeps the `SERIAL` counter	The same – **unless** you add `RESTART IDENTITY`
Can take a `WHERE`	No `WHERE` – always every row

Both can be undone in PostgreSQL: `BEGIN; TRUNCATE ...; ROLLBACK;` works, just like with `DELETE`.

INDEX: NEED 3

The need	Real scenarios	What the database provides	Read more
3.1 Remove the rows you name	Two never-borrowed copies of *Guards! Guards!*	`DELETE ... WHERE` `DELETE FROM Copy WHERE book_id = 6;`	DBS 3.9
3.2 Delete or mark?	Withdrawn books with years of loans	mark instead of delete; foreign keys may block a delete `UPDATE Copy SET status = 'withdrawn' WHERE ...;`	DBS 3.9
3.3 Empty a whole table	Last week's import table	`TRUNCATE` vs `DELETE`; numbering `TRUNCATE TABLE t RESTART IDENTITY;`	PG `TRUNCATE`

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (3.9 = chapter 3, section 9); PG = the PostgreSQL documentation, page for that command.

Always: wrap hand-run `UPDATE`/`DELETE`/`TRUNCATE` in `BEGIN ... ROLLBACK;` first, confirm the row count, **then** `COMMIT`.

NEED 4: SOMETHING LEAVES THAT OTHERS DEPEND ON

A member, Freja, leaves the library. But other records still point to her: her loans, and Maria, whom she referred.

SQL: the five `ON DELETE` actions

NEED 4: SOMETHING LEAVES THAT OTHERS DEPEND ON

The hardest case: a record leaves, but other records still point to it.

WHAT THE DATABASE CAN PROVIDE FOR YOU

1. Stop and tell you

The delete stops; the error says who still points at it.

2. Delete the dependants too

The rows that point at it are deleted with it.

3. Clear the reference

The dependants stay; their reference becomes empty.

4. Point it elsewhere

The dependants stay; their reference gets a default value.

WHAT YOU MUST DECIDE

1. A loan is history

Stop, so a person decides what happens to it.

2. A genre tag needs its book

Delete the tags together with the book.

3. A referral is only a note

Clear it: the member stays.

4. You want to record “deleted”

Point it at a placeholder row, member 0.

IN THE LIBRARY: A REAL REFERRAL CHAIN

Freja (11) asks to have her account deleted. But Maria (21) was referred by her – and the chain goes on. What happens to Maria? The same four answers:

member_id	name	referred_by
1	Daniel Petersen	NULL – founder
11	Freja Andersen	1
21	Maria Andersen	11
31	Maria Poulsen	21
41	Peter Olsen	31

1. **Stop and tell you.** Freja stays; the error names Maria.
2. **Delete them too.** Maria goes – and everyone below her.
3. **Clear it.** Maria stays, referred by nobody.
4. **Point elsewhere.** Maria points at member 0, “(deleted)”.

WHAT THE DATABASE OFFERS FOR NEED 4

	Need	SQL	What it does
4.1	Stop and tell you	`RESTRICT` / `NO ACTION`	Stops the delete while anything still points at the row, and the error says what
4.2	Delete the dependants too	`CASCADE`	Deletes every row that points at it – and the rows that point at those
4.3	Clear the reference	`SET NULL`	Keeps the dependants and empties their reference
4.4	Point it elsewhere	`SET DEFAULT`	Sets the reference to the column's default value

You set the rule once per foreign key, in the schema – `ON DELETE ...` – and the database applies it on every delete.

What it cannot do for you: pick the rule. All four are legal, and the wrong one silently erases a referral chain.

NO ACTION: TODAY'S DEFAULT

```
BEGIN;
ALTER TABLE Member DROP CONSTRAINT member_referred_by_fkey;
ALTER TABLE Member ADD CONSTRAINT member_referred_by_fkey
    FOREIGN KEY (referred_by) REFERENCES Member(member_id);
    -- no ON DELETE clause: NO ACTION
DELETE FROM Loan WHERE member_id = 11;
DELETE FROM Member WHERE member_id = 11;
```

```
DELETE 5
ERROR:  update or delete on table
  "member" violates foreign key
  constraint "member_referred_by_fkey"
  on table "member"
DETAIL:  Key (member_id)=(11) is
  still referenced from table
  "member".
```

Read it as: *put back Lecture 5's rule (no `ON DELETE`), clear Freja's loans, then delete Freja (11).*

- **Stopped – and told why.** The error names the table that still points at Freja. You decide what next.
- `ROLLBACK;` puts everything back, the 5 loans too.

ON DELETE CASCADE

```
-- clear the whole chain's loans first
BEGIN;
DELETE FROM Loan WHERE member_id IN (11,21,31,41);
ALTER TABLE Member DROP CONSTRAINT member_referred_by_fkey;
ALTER TABLE Member ADD CONSTRAINT member_referred_by_fkey
    FOREIGN KEY (referred_by) REFERENCES Member(member_id)
    ON DELETE CASCADE;
DELETE FROM Member WHERE member_id = 11;
SELECT member_id FROM Member WHERE member_id IN (11,21,31,41);
```

```
 member_id
-----------
(0 rows)
```

Read it as: *the same delete, but now with `ON DELETE CASCADE`.*

- **Deleting one member deleted four.** 21, 31 and 41 went with Freja – the whole chain, without a warning.
- `ROLLBACK;` brings all four back.

ON DELETE SET NULL

```
BEGIN;
DELETE FROM Loan WHERE member_id = 11;
ALTER TABLE Member DROP CONSTRAINT member_referred_by_fkey;
ALTER TABLE Member ADD CONSTRAINT member_referred_by_fkey
    FOREIGN KEY (referred_by) REFERENCES Member(member_id)
    ON DELETE SET NULL;
DELETE FROM Member WHERE member_id = 11;
SELECT member_id, referred_by FROM Member WHERE member_id = 21;
```

```
 member_id | referred_by
-----------+-------------
        21 |
(1 row)
```

Read it as: *the same delete, now with `ON DELETE SET NULL`.*

- **Freja is gone; Maria stays.** Only Maria's `referred_by` becomes empty. 31 and 41 are untouched.
- `ROLLBACK;` brings Freja back.

ON DELETE SET DEFAULT

```
BEGIN;
DELETE FROM Loan WHERE member_id = 11;
INSERT INTO Member (member_id, first_name, last_name, email)
VALUES (0, '(deleted)', '(deleted)', 'deleted@example.dk');
ALTER TABLE Member ALTER COLUMN referred_by SET DEFAULT 0;
ALTER TABLE Member DROP CONSTRAINT member_referred_by_fkey;
ALTER TABLE Member ADD CONSTRAINT member_referred_by_fkey
    FOREIGN KEY (referred_by) REFERENCES Member(member_id)
    ON DELETE SET DEFAULT;
DELETE FROM Member WHERE member_id = 11;
SELECT member_id, referred_by FROM Member WHERE member_id = 21;
```

```
 member_id | referred_by
-----------+-------------
        21 |           0
(1 row)
```

Read it as: *create member 0, “(deleted)”, make it the default – then the same delete.*

- **Maria is now “referred by” member 0** – a placeholder: her referrer was deleted.
- **Member 0 must exist,** or the foreign key refuses the delete. `ROLLBACK;`

RESTRICT VS. NO ACTION

```
REFERENCES Member(member_id) ON DELETE RESTRICT     -- checks at once, always
REFERENCES Member(member_id) ON DELETE NO ACTION    -- checks at once, by default ...
    DEFERRABLE INITIALLY DEFERRED                   -- ... or only at COMMIT
```

Read it as: *two ways to stop the delete – the only difference is when they check.*

- `RESTRICT` **checks at once, always.** `Loan`'s references to `Member` and `Copy` use it.
- `NO ACTION` **can wait until** `COMMIT`. That helps when a transaction breaks a reference and fixes it again before it ends. It is the default – Lecture 5's `referred_by` used it.

TRUNCATE AND FOREIGN KEYS

```
TRUNCATE TABLE Book;
```

```
ERROR:  cannot truncate a table referenced in a foreign key constraint
DETAIL:  Table "bookgenre" references "book".
HINT:  Truncate table "bookgenre" at the same time, or use TRUNCATE ... CASCADE.
```

Read it as: *empty the whole `Book` table – refused, although its copies and tags are `ON DELETE CASCADE`.*

- **`TRUNCATE` ignores the `ON DELETE` rules.** It does not look row by row. If any table points at this one, it refuses – whatever rule that foreign key has.
- **The way out:** `TRUNCATE Book CASCADE;` empties every table that points at `Book`, and the tables that point at those – here `BookGenre`, `Copy`, `BookAuthor`, and through `Copy` also `Loan`. Powerful, and dangerous.

INDEX: NEED 4

The need	Real scenarios	What the database provides	Read more
4.1 Stop and tell you	Freja still has loans on record	`ON DELETE RESTRICT`: history beats convenience `REFERENCES Member(member_id) ON DELETE RESTRICT` (as `Loan` has)	DBS 4.4
4.2 Delete the dependants	A book is withdrawn, its copies with it	`ON DELETE CASCADE` `REFERENCES Book(book_id) ON DELETE CASCADE`	DBS 4.4
4.3 Clear the reference	Freja leaves; Maria was referred by her	`ON DELETE SET NULL` `FOREIGN KEY (referred_by) REFERENCES Member(member_id) ON DELETE SET NULL`	DBS 4.4
4.4 Point it elsewhere	Maria's referrer becomes member 0, "(deleted)"	`ON DELETE SET DEFAULT`, with a placeholder row `... DEFAULT 0 REFERENCES Member(member_id) ON DELETE SET DEFAULT`	DBS 4.4
Change the rule later	Switch `referred_by` to `SET NULL`	`ALTER TABLE ... DROP/ADD CONSTRAINT` `ALTER TABLE Member ADD CONSTRAINT ... FOREIGN KEY ...;`	PG `ALTER TABLE`

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (3.9 = chapter 3, section 9); PG = the PostgreSQL documentation, page for that command.

EFFICIENCY: GIVING A QUESTION A NAME

Not about changing data – a small tool that saves work: `CREATE VIEW`

CREATE VIEW: A SAVED QUERY AS A VIRTUAL TABLE

```sql
CREATE VIEW ActiveLoan AS
SELECT l.loan_id, m.first_name, m.last_name, b.title,
       l.loan_date, l.due_date
FROM Loan l
JOIN Member m ON l.member_id = m.member_id
JOIN Book b ON l.book_id = b.book_id
WHERE l.return_date IS NULL;
```

Read it as: *save this question under the name `ActiveLoan`: every loan not yet returned – who has which book.*

- **A view stores the question, not the answers.** Every `SELECT * FROM ActiveLoan` runs the join again, on today's data.
- **Use it like a table.** `WHERE`, `ORDER BY` and further joins all work on it.
- **Why bother:** a long join gets a short name, and a question you ask often lives in one place.

WORKED EXAMPLE: ACTIVELOAN

```sql
SELECT first_name, last_name, title
FROM ActiveLoan
ORDER BY due_date
LIMIT 3;
```

first_name	last_name	title
Marcus	Hansen	Of Mice and Men
Anna	Nielsen	The Handmaid's Tale
Clara	Olsen	Childhood's End

Read it as: *the three loans that are due first.*

- **One short line instead of a three-table join.** The join and the `return_date IS NULL` filter are written once, inside the view – 63 loans are open right now.
- **The answer follows the data.** Return a book, and it drops out of `ActiveLoan` at once. (The due dates depend on the day the database was loaded, so they are not shown.)

WORKED EXAMPLE: BOOKAVAILABILITY

```sql
CREATE VIEW BookAvailability AS
SELECT b.book_id, b.title,
  COUNT(*) AS total_copies,
  COUNT(*) FILTER (WHERE c.status = 'available')
    AS available_copies
FROM Book b JOIN Copy c ON b.book_id = c.book_id
GROUP BY b.book_id, b.title;

SELECT * FROM BookAvailability
WHERE available_copies = 0;
```

title	total	available
American Gods	2	0
Animal Farm	1	0
Atomic Habits	1	0
… 20 books in all		

Read it as: *for every book: how many copies it has, and how many are on the shelf.*

- `COUNT(*) FILTER (WHERE ...)` **counts only some rows** – here, the available copies. No subquery needed.
- **The morning question becomes one line:** “which books have no copy on the shelf?”

VIEWS: WHAT THEY ARE NOT

- **Not a copy.** Every query runs the view's question again. If the join is slow, the view is just as slow.
- **Not always writable.** A simple view over one table can be updated. A view with joins or counts cannot – not without extra work.
- **Not a stored snapshot.** *Materialized* views do store a result and must be refreshed – that is a different tool, not covered here.
- **What a view is: a name.** The morning question becomes `SELECT * FROM BookAvailability`. When the question changes, you change it in one place.

WRAP-UP

LECTURE 6 IN NEED ASPECT: WHAT THE WORLD NEEDS

	Need	What the database can provide	What you must decide
1	Something new arrives	**1.1** Add one record **1.2** Add many at once	**1.3** Already there: whose data wins? **1.4** Unclear data: never guess
2	Something changes	**2.1** Change the rows you name **2.2** Rows found through another table	**2.3** Undo a mistake: check the count first
3	Something leaves	**3.1** Remove the rows you name **3.3** Empty a whole table	**3.2** Delete it, or only mark it?
4	Something leaves that others depend on	**4.1** Stop and tell you **4.2** Delete them too **4.3** Clear the reference **4.4** Point elsewhere	Which rule fits each reference?

These four steps are the life of any record, in any system – not just PostgreSQL. The right-hand column stays your decision.

LECTURE 6 IN EXAMPLE ASPECT: HOW THE LIBRARY MET THEM

	In the library
1	**1.1** Lisa gets a card, and her number comes back at once. **1.2** Every book from before 1950 is tagged Classic in one step. **1.3** Anna is already ours: we decided whose data wins. **1.4** The 2009 file: `03/04/2024` is set aside and sent back, not guessed.
2	**2.1** Copy 1 of *Dune* is marked lost. **2.2** Every copy of *Dune*, found by its title. **2.3** A missing `WHERE` marks all 157 copies lost – and `ROLLBACK` takes it back.
3	**3.1** The two never-borrowed copies of *Guards! Guards!* are deleted. **3.2** *Dune* is withdrawn: marked, not deleted – its 6 loans are history. **3.3** Last week's import table is emptied.
4	Freja leaves, and Maria was referred by her. **4.1** Stopped: the error says Maria points at her. **4.2** The whole chain is deleted. **4.3** Maria stays, referred by nobody – the right answer for a referral. **4.4** Maria points at member 0, "(deleted)".

The library is how you will recognise each need next time – in a hospital, a shop, a ship's log.

LECTURE 6 IN PRACTICE ASPECT: WHAT THE DATABASE OFFERS

	What the database can provide	What you must decide
1	**1.1** `INSERT` **1.2** multi-row `INSERT`, `INSERT ... SELECT`, `COPY` **1.1 & 1.2** `SERIAL`, `RETURNING`	**1.3** `ON CONFLICT DO NOTHING` / `DO UPDATE` **1.4** `COPY ... WITH (ENCODING ..., NULL ...)`
2	**2.1** `UPDATE ... SET ... WHERE` **2.2** `UPDATE ... FROM`	**2.3** `BEGIN ... COMMIT` or `ROLLBACK`
3	**3.1** `DELETE ... WHERE` **3.3** `TRUNCATE (RESTART IDENTITY)`	**3.2** blocked `DELETE` ⟶ `UPDATE ... SET status`
4	**4.1** `RESTRICT` / `NO ACTION` **4.2** `CASCADE` **4.3** `SET NULL` **4.4** `SET DEFAULT`	the `ON DELETE` rule you choose, per foreign key

The SQL is the part that changes with the tool. The needs and the decisions on page 1 stay.

WHERE TO READ MORE

Concept	Where
`INSERT`/`UPDATE`/`DELETE`; `INSERT ... SELECT`	DBS ch. 3.9 *Modification of the Database*
`SERIAL`, defaults, generated keys	DBS ch. 4.5 *SQL Data Types and Schemas*
Referential integrity, `ON DELETE` actions	DBS ch. 4.4 *Integrity Constraints*; PostgreSQL docs “Constraints”
Views (efficiency)	DBS ch. 4.2 *Views*; PostgreSQL docs `CREATE VIEW`
`COPY`, `ON CONFLICT`, `TRUNCATE`, `RETURNING`	PostgreSQL reference: `COPY`, `INSERT`, `TRUNCATE`
`BEGIN`, `COMMIT`, `ROLLBACK`	DBS ch. 4.3 *Transactions*; the full story (ACID, isolation) is next lecture, DBS ch. 17

DBS = *Database System Concepts*, Silberschatz, Korth & Sudarshan, 7th ed. (3.9 = chapter 3, section 9); PostgreSQL docs = the official PostgreSQL documentation, page for that command.

EXERCISES

	Need	Ex.	What you do – exercise sheet 6
1	Something new arrives	1, 2	Add an author and copies; load the city's list with `ON CONFLICT`
2	Something changes	3	Member 8's grace week: predict the count, then the missing `WHERE`
3	Something leaves	4	Return a loan; delete or mark a copy; `DELETE` vs `TRUNCATE`
4	Something leaves that others depend on	5	Delete books and members: predict, then check in `BEGIN ... ROLLBACK`
1	A file from outside	6	The old 2009 file: fix what is clear, send back the rest
	Efficiency	7	A view `BookAvailability`: which books have no copy on the shelf?
	Your own project	8	One script; an `ON DELETE` rule per foreign key, and why; one upsert; one view

Every statement comes with three lines: **1.** which Index row it belongs to **2.** why it is right for your schema **3.** how you checked it – a query, a row count, a `BEGIN ... ROLLBACK`.

SUMMARY

Lecture 6 – changing data safely, along the life of a record:

1. **It arrives.** `INSERT`, `COPY`; `SERIAL` and `RETURNING` help. You decide whose data wins – and never guess.
2. **It changes.** `UPDATE ... WHERE`. `BEGIN` first, check the count, then `COMMIT` or `ROLLBACK`.
3. **It leaves.** `DELETE ... WHERE` – or mark it, when its history matters. `TRUNCATE` empties a table.
4. **It leaves while others depend on it.** One `ON DELETE` rule per foreign key, applied on every delete.

Efficiency: `CREATE VIEW` gives a question a name.

Next lecture – keeping data right when things get complex: a task with several steps, many people writing at once, and data that grows large – and the four ACID promises that keep it stable and accurate.